

Оглавление

1. Цель и смысл итоговой программы.....	3
2. Общая логика обработки данных.....	3
3. Основные константы программы.....	4
4. Класс TextPreprocessor.....	5
5. Класс PathResolver.....	6
6. Класс GlossaryBuilder.....	6
7. Класс ArticleStatsExtractor.....	7
8. Класс StatementDatasetBuilder.....	8
8.1. Поиск и фильтрация утверждений.....	8
8.2. Добавление контекста.....	8
8.3. Классификация утверждений.....	9
8.4. Формальная запись и Lean 4.....	9
8.5. Формирование записи датасета.....	10
9. Класс AnnotationAgreementCalculator.....	10
10. Функция run_tasks_9_1_9_2_and_dataset().....	11
11. Функция main() и запуск программы.....	11
12. Выходные файлы программы.....	12
13. Соответствие требованиям работы.....	12
14. Логика развития кода.....	13
15. Проверка работоспособности и устойчивость.....	14
Заключение.....	14

1. Цель и смысл итоговой программы

В данной работе разработана программа, которая автоматизирует подготовку датасета для задачи автоформализации математических утверждений на русском языке. Основная идея заключается в том, чтобы взять набор PDF-файлов с математическими публикациями, извлечь из них текст, очистить его от технических фрагментов, найти предложения, похожие на математические утверждения, классифицировать их и сохранить результат в структурированном виде.

Отчёт построен не как описание отдельных коммитов, а как пояснение финального кода: что именно в нём реализовано, зачем это нужно и какую роль каждый блок играет в общем процессе. Такой формат удобнее для защиты, потому что показывает не только наличие кода, но и понимание его структуры.

Итоговая программа решает практическую задачу: из неструктурированных PDF-документов формируется набор записей, пригодный для дальнейшей ручной проверки, разметки и использования в экспериментах по переходу от естественного математического языка к формальным заготовкам Lean 4.

2. Общая логика обработки данных

Финальный код реализует последовательный конвейер обработки. Каждый этап получает результат предыдущего этапа и подготавливает данные для следующего. Благодаря этому программа остаётся читаемой: извлечение текста, очистка, поиск утверждений, классификация, генерация формальных полей и экспорт не смешиваются в одном методе.

Этап	Что происходит	Зачем нужен этап
1. Чтение PDF	Программа находит все PDF-файлы в папке pdfs и извлекает из них текст.	PDF является исходным форматом публикаций, поэтому сначала нужно получить машинно обрабатываемый текст.
2. Очистка текста	Удаляются переносы, лишние	Это снижает шум и не даёт

	пробелы, служебные символы, начало статьи и список литературы.	заголовкам, аннотациям и библиографии попадать в датасет как математические утверждения.
3. Глоссарий и статистика	Формируются glossary.json, notation_table.csv и article_stats.csv.	Эти файлы показывают, какие алгебраические термины и обозначения встречаются в корпусе.
4. Поиск кандидатов	Текст разбивается на предложения; выбираются фрагменты с математическими маркерами и ключевыми словами.	Так программа находит фразы, похожие на теоремы, леммы, определения или утверждения.
5. Добавление контекста	К найденному предложению добавляется соседнее предложение до и после.	Математическая формулировка часто зависит от контекста, поэтому одно предложение может быть недостаточным.
6. Классификация	Утверждения распределяются по типам ISOMORPH, SUBSTRUCTURE, ACTION, CLASSIFICATION, PROPERTY.	Тип утверждения нужен для тематической разметки и дальнейшего анализа качества датасета.
7. Генерация полей	Создаются русский текст, английская версия, формальная запись, Lean-шаблон, ключевые слова, тип и источник.	Эти поля соответствуют структуре итогового датасета.
8. Экспорт	Результаты сохраняются в JSON, CSV, XLSX и файл для аннотации.	Разные форматы нужны для анализа, просмотра в Excel и ручной проверки аннотаторами.

PDF → clean text → term statistics → statement candidates → classification → dataset records → JSON / CSV / XLSX / annotation file

3. Основные константы программы

В начале кода заданы словари и списки, которые определяют предметную область и структуру результата. Они вынесены отдельно, потому что именно эти данные чаще всего нужно изменять при адаптации программы под другую математическую тему.

Элемент кода	Назначение	Почему он нужен
GLOSSARY	Ручной глоссарий базовых алгебраических объектов: группа, подгруппа, кольцо, идеал, поле, модуль, алгебра, гомоморфизм, изоморфизм.	Используется как теоретическая основа и как справочный блок для построения glossary.json.

KEY_TERMS	Набор регулярных выражений для поиска русских и английских терминов.	Позволяет находить тематически релевантные фрагменты в статьях.
NOTATION_PATTERNS	Шаблоны для поиска математических обозначений: G, H, K, R, S, F, M, N, A, B.	Нужны для таблицы обозначений и оценки того, какие символы чаще встречаются в корпусе.
DATASET_FIELDS	Фиксированный список столбцов итогового датасета.	Гарантирует одинаковую структуру JSON, CSV и XLSX-файлов.

4. Класс TextPreprocessor

Класс TextPreprocessor отвечает за предварительную обработку текста. Он вынесен в отдельный класс, потому что очистка текста используется в нескольких частях программы: при извлечении статистики, при подготовке txt-файлов и при формировании записей для Excel.

Метод	Что делает	Зачем реализован
remove_excel_illegal_chars()	Удаляет управляющие символы диапазонов \x00-\x08, \x0b-\x0c, \x0e-\x1f.	Некоторые PDF содержат невидимые символы, из-за которых openpyxl не может сохранить XLSX. Метод предотвращает ошибку IllegalCharacterError.
limit_excel_cell_length()	Ограничивает длину строки до допустимого размера ячейки Excel.	Excel имеет ограничение на длину текста в одной ячейке, поэтому слишком длинные фрагменты нужно обрезать.
safe_text()	Последовательно применяет очистку от запрещённых символов и ограничение длины.	Используется как универсальная защита перед записью данных в Excel и CSV.
clean_text()	Убирает мягкие переносы, лигатуры, лишние пробелы, переносы строк внутри предложений.	Текст из PDF часто содержит технические разрывы, которые мешают поиску предложений и терминов.
remove_front_and_back_matter()	Удаляет начало статьи и заключительную часть со списком литературы.	В датасет должны попадать математические фрагменты, а не библиография и служебная информация.
split_sentences()	Разбивает очищенный текст на предложения подходящей длины.	Нужно для поиска отдельных утверждений-кандидатов.

Этот класс является технической основой всей программы. Без него итоговый датасет был бы загрязнён PDF-артефактами, а сохранение Excel-файла могло бы завершаться ошибкой на разных компьютерах.

5. Класс PathResolver

Класс PathResolver добавлен для корректной работы с путями. На практике код может запускаться из корня проекта или из папки pdfs, поэтому относительные пути не всегда интерпретируются одинаково. PathResolver определяет расположение скрипта и находит папки pdfs и output более устойчиво.

Метод	Роль
resolve_pdf_dir()	Определяет, где находится папка с PDF-файлами, даже если программа запущена из другой рабочей директории.
resolve_output_dir()	Определяет, куда сохранять результаты, чтобы output создавался в ожидаемом месте проекта.

Этот блок нужен не для математической обработки, а для надёжности запуска. Благодаря ему программа меньше зависит от того, из какой папки пользователь нажал Run или выполнил команду `python main.py`.

6. Класс GlossaryBuilder

GlossaryBuilder развивает исходную идею глоссария из первого варианта кода. Он не только хранит базовые определения, но и ищет в PDF примеры употребления терминов и обозначений.

Метод	Назначение
extract_text_with_context()	Извлекает текстовые блоки из PDF вместе со страницей и координатами блока. Это нужно для сохранения контекста найденных терминов.
find_object_examples()	Ищет в текстовых блоках примеры употребления конкретного математического объекта.
find_notations()	Подсчитывает встречаемость типовых обозначений для групп, колец, полей, модулей и алгебр.

<code>build_table_from_pdfs()</code>	Обходит все PDF-файлы, собирает примеры, формирует <code>glossary.json</code> и <code>notation_table.csv</code> .
--------------------------------------	---

Смысл этого класса — показать, какие объекты и обозначения реально встречаются в исходных публикациях. Это полезно перед формированием датасета, потому что подтверждает тематическую релевантность корпуса.

7. Класс `ArticleStatsExtractor`

`ArticleStatsExtractor` отвечает за автоматическое извлечение текста и статистику терминов. В отличие от `GlossaryBuilder`, который работает с примерами и обозначениями, этот класс формирует чистые текстовые версии статей и количественную сводку.

Метод	Что делает	Практический смысл
<code>extract_text_from_pdf()</code>	Постранично извлекает текст из PDF.	Сохраняет связь текста с исходной статьёй и позволяет обрабатывать несколько файлов.
<code>postprocess_text()</code>	Применяет очистку текста и удаление служебных частей.	Готовит материал для дальнейшего поиска утверждений.
<code>count_terms()</code>	Подсчитывает частоты ключевых математических терминов.	Позволяет оценить насыщенность статей нужной тематикой.
<code>process_articles()</code>	Создаёт <code>txt_clean/*.txt</code> и <code>article_stats.csv</code> .	Формирует промежуточный корпус очищенных текстов, который потом использует <code>StatementDatasetBuilder</code> .

Файлы в папке `txt_clean` являются важным промежуточным результатом. Они позволяют отделить этап извлечения PDF от этапа построения датасета. Если нужно проверить качество извлечения, можно открыть эти `txt`-файлы и увидеть, что именно программа дальше анализирует.

8. Класс StatementDatasetBuilder

StatementDatasetBuilder является центральным классом программы. Именно он превращает очищенный текст в записи итогового датасета. Логика класса разделена на небольшие методы: поиск ключевых слов, определение кандидатов, добавление контекста, классификация, генерация формальных полей и сохранение результатов.

8.1. Поиск и фильтрация утверждений

Метод `detect_keywords()` ищет математические термины из KEY_TERMS. Метод `has_statement_marker()` проверяет наличие слов-маркеров: теорема, лемма, следствие, proposition, theorem, definition и других. Метод `is_statement_candidate()` объединяет эти признаки: предложение считается кандидатом, если в нём есть тематический термин и либо маркер математического утверждения, либо логические конструкции вида «если», «то», «существует», а также символы $\forall$, $\exists$, $\Rightarrow$, $\Leftrightarrow$, $\cong$, $\leq$, $\subset$, $\subseteq$, $\supset$.

Такой подход является правилowym, а не нейросетевым. Его преимущество — воспроизводимость: можно объяснить, почему конкретный фрагмент попал в датасет. Недостаток — часть сложных утверждений может быть пропущена, а часть нерелевантных фраз может попасть в кандидаты.

8.2. Добавление контекста

Метод `make_context_statement()` добавляет к найденному предложению одно предыдущее и одно следующее предложение, если они не слишком длинные. Это было реализовано потому, что в математических статьях утверждение часто зависит от введённых перед ним обозначений или условий. Без контекста запись может быть неполной.

основное предложение-кандидат
+ предыдущее предложение, если оно короткое
+ следующее предложение, если оно короткое

8.3. Классификация утверждений

Метод `classify_statement()` относит найденный фрагмент к одному из пяти типов: `ISOMORPH`, `SUBSTRUCTURE`, `ACTION`, `CLASSIFICATION`, `PROPERTY`. Для каждого типа задан набор регулярных выражений. Программа считает совпадения и выбирает тип с наибольшим числом найденных признаков.

Тип	Что означает	Примеры признаков
ISOMORPH	Утверждения об изоморфизме или эквивалентности структур.	изоморф, isomorph, equivalent
SUBSTRUCTURE	Утверждения о подструктурах: подгруппах, идеалах, подмодулях.	подгрупп, идеал, subgroup, ideal
ACTION	Утверждения о действиях, орбитах, стабилизаторах и представлениях.	действие, орбита, stabilizer, action
CLASSIFICATION	Утверждения классификационного характера.	классификация, класс, characterization
PROPERTY	Утверждения о свойствах алгебраических объектов.	свойство, коммутативный, абелев, finite, simple

8.4. Формальная запись и Lean 4

Метод `make_formal_statement()` создаёт обобщённую формальную запись в математической нотации. Метод `make_lean_code()` формирует шаблон Lean 4 + Mathlib. Эти поля не являются полным доказательством конкретной теоремы из статьи. Они нужны как заготовка для будущей формализации и как демонстрация связи между естественным текстом, формальной записью и Lean-представлением.

Например, если утверждение классифицировано как `ISOMORPH`, создаётся шаблон про наличие изоморфизма между группами. Если утверждение относится к `SUBSTRUCTURE`, генерируется шаблон для подгруппы, идеала или подмодуля в зависимости от найденных ключевых слов.

```
theorem dataset_item_0001 {G H : Type*} [Group G] [Group H] :  
  Nonempty (G  $\simeq$  H)  $\rightarrow$  Nonempty (H  $\simeq$  G) := by  
  intro h
```



```
rcases h with ⟨e⟩
exact ⟨e.symm⟩
```

8.5. Формирование записи датасета

Метод `make_record()` собирает одну строку итогового датасета. В неё входят семь основных полей: русская формулировка, английская формулировка, формальная запись, код Lean 4 + Mathlib, ключевые слова, тип утверждения и источник.

Поле датасета	Что содержит
Формулировка на естественном языке (русский)	Извлечённый из статьи фрагмент с контекстом.
Формулировка на естественном языке (английский)	Шаблонный перевод на английский через словарь замен.
Запись на формальном языке	Обобщённая логическая форма утверждения.
Код Lean 4 + Mathlib	Lean-заготовка, соответствующая типу утверждения.
Ключевые слова (русский / английский)	Список найденных терминов.
Тип утверждения	Результат классификации: ISOMORPH, SUBSTRUCTURE, ACTION, CLASSIFICATION или PROPERTY.
Источник	Имя txt-файла, полученного из исходного PDF.

Перед сохранением запись проходит через `sanitize_record()`. Это нужно, чтобы убрать запрещённые символы и избежать ошибок при записи в Excel.

9. Класс `AnnotationAgreementCalculator`

Для контроля качества разметки в программе реализован класс `AnnotationAgreementCalculator`. Он рассчитывает коэффициент согласия аннотаторов Cohen's kappa по колонкам `annotator_1` и `annotator_2` из файла `annotation_pairs.csv`.

Смысл этого блока заключается в том, что датасет не должен зависеть только от автоматической классификации. Два человека могут независимо разметить типы утверждений, после чего программа вычислит степень их

согласия. Целевым ориентиром в постановке задачи является значение κ не ниже 0,70.

Метод	Назначение
<code>cohen_kappa()</code>	Вычисляет коэффициент согласия между двумя списками меток.
<code>calculate_from_file()</code>	Читает <code>annotation_pairs.csv</code> , извлекает метки двух аннотаторов и сохраняет результат в <code>annotation_agreement.json</code> .

10. Функция `run_tasks_9_1_9_2_and_dataset()`

Функция `run_tasks_9_1_9_2_and_dataset()` объединяет все этапы в один запуск. Она создаёт выходную папку, строит глоссарий и таблицу обозначений, извлекает статистику из статей, собирает датасет, сохраняет файлы и формирует служебный `report.json`.

```
run_tasks_9_1_9_2_and_dataset()
→ GlossaryBuilder
→ ArticleStatsExtractor
→ StatementDatasetBuilder
→ AnnotationAgreementCalculator
→ report.json
```

Наличие единой функции запуска упрощает использование программы: пользователю не нужно вручную вызывать отдельные классы. Достаточно положить PDF-файлы в папку `pdfs` и выполнить `python main.py`.

11. Функция `main()` и запуск программы

Функция `main()` отвечает за запуск из командной строки. Через `argparse` можно передать путь к папке с PDF и путь к выходной папке. Если параметры не указаны, используются значения по умолчанию: `pdfs` и `output`.

```
python main.py
python main.py --pdf_dir pdfs --output_dir output
```

Для работы программы нужны библиотеки `PyMuPDF`, `pandas` и `openpyxl`. `PyMuPDF` используется для чтения PDF, `pandas` — для формирования таблиц, `openpyxl` — для записи XLSX-файла.

```
python -m pip install --upgrade pymupdf pandas openpyxl
```

12. Выходные файлы программы

После запуска программа формирует несколько файлов. Они разделены по назначению: часть нужна для контроля качества извлечения текста, часть — для статистики, часть — для итогового датасета и ручной аннотации.

Файл или папка	Назначение
output/txt_clean/*.txt	Очищенные текстовые версии исходных PDF-статей.
output/glossary.json	Глоссарий алгебраических структур и примеры употребления терминов из PDF.
output/notation_table.csv	Таблица найденных обозначений и числа их упоминаний.
output/article_stats.csv	Постатейная статистика по ключевым терминам.
output/dataset.json	Полный итоговый датасет в структурированном формате.
output/dataset_50.csv	CSV-версия первых 50 записей для просмотра и проверки.
output/Dataset.xlsx	Excel-файл с первыми 30 строками датасета.
output/annotation_pairs.csv	Файл для ручной разметки двумя аннотаторами.
output/annotation_agreement.json	Результат расчёта Cohen's kappa после заполнения аннотаций.
output/report.json	Технический отчёт о количестве обработанных статей, записей и путях к файлам.

13. Соответствие требованиям работы

Финальная версия кода закрывает основные практические требования: извлечение текста из PDF, очистка статей, выделение математических фрагментов, классификация, подготовка полей датасета, экспорт в JSON/CSV/XLSX и создание файла для ручной аннотации.

Требование	Как реализовано в коде
Извлечение структурированного текста из PDF	Используется fitz/PyMuPDF; дополнительно сохраняются очищенные txt-файлы.
Сегментация статьи на блоки	Текст очищается и разбивается на предложения; кандидаты расширяются соседними предложениями.

Разметка математических утверждений	Используются ключевые термины, маркеры утверждений и логические символы.
Классификация утверждений	Метод <code>classify_statement()</code> распределяет фрагменты по пяти типам.
Пары для генеративных задач	Каждая запись содержит русский текст, английский вариант, формальную запись и Lean-шаблон.
Парная аннотация	Создаётся <code>annotation_pairs.csv</code> с колонками <code>annotator_1</code> и <code>annotator_2</code> .
Расчёт согласия	Реализован Cohen's kappa в <code>AnnotationAgreementCalculator</code> .
Итоговый JSON-датасет	Формируется <code>dataset.json</code> , а также CSV и XLSX-представления.

При этом важно понимать границы реализации. Программа не выполняет полноценный машинный перевод и не доказывает извлечённые теоремы в Lean автоматически. Английская формулировка и Lean-код являются шаблонными заготовками. Это допустимо для практической работы, потому что цель состоит в подготовке структуры датасета и демонстрации конвейера обработки, а не в полном решении задачи автоформализации.

14. Логика развития кода

Изначально код содержал глоссарий и подсчёт терминов. В финальной версии он был расширен до полноценного конвейера подготовки датасета. Логика развития может быть представлена через последовательность этапов разработки.

Этап	Что было добавлено
1	Базовый обработчик PDF и глоссарий алгебраических структур.
2	Отдельный класс предобработки текста.
3	Удаление служебных частей научных статей.
4	Фиксированная структура полей датасета.
5	Извлечение математических утверждений из текста.
6	Расширение найденных утверждений соседним

	контекстом.
7	Классификация утверждений по типам.
8	Генерация формальных записей и Lean 4 шаблонов.
9	Формирование одной записи датасета.
10	Финальная сборка датасета, экспорт, аннотация, Cohen's kappa и исправление Excel-ошибок.

15. Проверка работоспособности и устойчивость

В код добавлены механизмы, которые повышают устойчивость программы при запуске на разных компьютерах. Во-первых, обрабатываются файлы с расширениями .pdf и .PDF, причём поиск выполняется рекурсивно. Во-вторых, PathResolver снижает риск ошибки при запуске из другой папки. В-третьих, очистка запрещённых символов решает проблему `openruhl IllegalCharacterError`, которая может возникать из-за невидимых управляющих символов в PDF.

Такая доработка важна, потому что программа должна работать не только на одном конкретном компьютере автора, но и на другом устройстве с теми же исходными PDF-файлами.

Заключение

В результате работы был получен финальный код программы, которая выполняет полный цикл подготовки датасета для задачи автоформализации математических утверждений на русском языке. Программа извлекает текст из PDF, очищает его, выделяет математические утверждения, классифицирует их по типам, формирует записи с русской и английской формулировкой, формальной записью, Lean 4-заготовкой, ключевыми словами и источником.

Главное достоинство реализации состоит в модульной структуре. Каждый класс отвечает за отдельный этап: `TextPreprocessor` — за очистку, `GlossaryBuilder` — за глоссарий и обозначения, `ArticleStatsExtractor` — за

статистику, `StatementDatasetBuilder` — за построение датасета, `AnnotationAgreementCalculator` — за проверку согласия аннотаторов. Благодаря этому код можно дорабатывать дальше: добавлять более точный перевод, расширять список математических терминов, улучшать классификацию и подключать настоящую генерацию Lean-формализаций.